

```
// Merged files from branch: main

// --- Start of pubspec.yaml ---
name: cl837_accelerometer
description: CL837 Accelerometer data reader
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  flutter_blue_plus: ^1.31.8
  permission_handler: ^11.0.1
  intl: ^0.18.0
  cupertino_icons: ^1.0.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^2.0.0

flutter:
  uses-material-design: true
// --- End of pubspec.yaml ---

// --- Start of lib\accelerometer_service.dart ---
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';
import 'battery.dart';
import 'heartrate.dart';

class SensorData {
  final DateTime timestamp;
  final double x;
  final double y;
  final double z;
  final int? heartRate;
  final int? batteryLevel;

  const SensorData({
    required this.timestamp,
    required this.x,
    required this.y,
    required this.z,
    this.heartRate,
    this.batteryLevel,
  });
}

class SensorService {
```

```

// Accelerometer Service & Characteristics
static const String _accelServiceUuid = 'aae28f00-71b5-42a1-8c3c-f9cf6ac969d0';
static const String _accelDataCharUuid = 'aae28f01-71b5-42a1-8c3c-f9cf6ac969d0';
static const double _scaleFactor = 8.0 / 32768.0; // For ±8 g range

final _dataStreamController = StreamController<SensorData>.broadcast();
final _stateStreamController = StreamController<bool>.broadcast();
StreamSubscription? _accelSubscription;
bool _isAccelerometerWorking = false;

Stream<SensorData> get dataStream => _dataStreamController.stream;
Stream<bool> get connectionStream => _stateStreamController.stream;
bool get isAccelerometerWorking => _isAccelerometerWorking;

void _processAccelData(List<int> value, int? heartRate, int? batteryLevel) {
  if (value.length < 3) return;
  try {
    if (value[0] != 0xFF) return;
    if (value[2] == 0x0c) {
      final now = DateTime.now();
      for (var i = 3; i < value.length - 1; i += 6) {
        if (i + 5 >= value.length) break;

        final rawX = _convertToSigned16(value[i] | (value[i + 1] << 8));
        final rawY = _convertToSigned16(value[i + 2] | (value[i + 3] << 8));
        final rawZ = _convertToSigned16(value[i + 4] | (value[i + 5] << 8));

        final x = rawX * _scaleFactor;
        final y = rawY * _scaleFactor;
        final z = rawZ * _scaleFactor;

        _dataStreamController.add(SensorData(
          timestamp: now,
          x: x,
          y: y,
          z: z,
          heartRate: heartRate,
          batteryLevel: batteryLevel,
        ));
      }
    }
  } catch (e) {
    debugPrint('Error processing accelerometer data: $e');
  }
}

int _convertToSigned16(int value) {
  value &= 0xFFFF;
  return (value & 0x8000) != 0 ? -(0x10000 - value) : value;
}

Future<void> _startAccelerometer(BluetoothDevice device) async {
  try {
    debugPrint('\n=== STARTING ACCELEROMETER SETUP ===');
  }
}

```

```

final services = await device.discoverServices();

// Setup Accelerometer
final accelService = services.firstWhere(
  (s) => s.uuid.toString().toLowerCase() == _accelServiceUuid.toLowerCase(),
  orElse: () => throw Exception('Accelerometer service not found'),
);

debugPrint('Found Accelerometer service: ${accelService.uuid}');
final accelChar = accelService.characteristics.firstWhere(
  (c) => c.uuid.toString().toLowerCase() == _accelDataCharUuid.toLowerCase(),
  orElse: () => throw Exception('Accelerometer characteristic not found'),
);

debugPrint('Found Accelerometer characteristic: ${accelChar.uuid}');
debugPrint('Accelerometer Properties: read=${accelChar.properties.read},
notify=${accelChar.properties.notify}');

// Enable notifications
final success = await accelChar.setNotifyValue(true);
debugPrint('Accelerometer notifications enabled: $success');

if (!success) {
  throw Exception('Failed to enable accelerometer notifications');
}

_accelSubscription = accelChar.lastValueStream.listen(
  (value) {
    debugPrint('Accelerometer data received: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
    _processAccelData(value, null, null); // Initial values without HR and
battery
  },
  onError: (error) {
    debugPrint('Accelerometer notification error: $error');
    _stateStreamController.add(false);
    _isAccelerometerWorking = false;
  },
);

_isAccelerometerWorking = true;
} catch (e) {
  debugPrint('Accelerometer setup error: $e');
  _isAccelerometerWorking = false;
  rethrow;
}
}

Future<void> _startServiceWithRetry(Future<void> Function() startFunction) async {
  const maxRetries = 3;
  for (var i = 0; i < maxRetries; i++) {
    try {
      await startFunction();
      return;
    }
  }
}

```

```

    } catch (e) {
      if (i == maxRetries - 1) {
        debugPrint('Service failed after $maxRetries retries: $e');
        rethrow;
      } else {
        debugPrint('Retrying service after error: $e');
        await Future.delayed(Duration(milliseconds: 500 * (i + 1)));
      }
    }
  }
}
}

```

```

Future<void> start(BluetoothDevice device, HeartRateService heartRateService,
  BatteryService batteryService) async {
  try {
    // Start accelerometer service first - this is critical
    await _startAccelerometer(device);

    // Start heart rate and battery services in parallel with error handling
    await Future.wait([
      _startServiceWithRetry(() => heartRateService.start(device))
        .catchError((e) {
          debugPrint('Heart rate service failed to start: $e');
          return null; // Continue without heart rate
        }),
      _startServiceWithRetry(() => batteryService.start(device))
        .catchError((e) {
          debugPrint('Battery service failed to start: $e');
          return null; // Continue without battery
        }),
    ], eagerError: false);

    // Set up subscriptions for heart rate and battery updates
    heartRateService.dataStream.listen((heartRate) {
      if (_accelSubscription != null && _isAccelerometerWorking) {
        // Update the sensor data with the new heart rate
        _dataStreamController.add(SensorData(
          timestamp: DateTime.now(),
          x: 0, // These will be updated by next accelerometer reading
          y: 0,
          z: 0,
          heartRate: heartRate,
          batteryLevel: batteryService.lastBatteryLevel,
        ));
      }
    }, onError: (e) => debugPrint('Heart rate stream error: $e'));

    batteryService.dataStream.listen((battery) {
      if (_accelSubscription != null && _isAccelerometerWorking) {
        // Update the sensor data with the new battery level
        _dataStreamController.add(SensorData(
          timestamp: DateTime.now(),
          x: 0, // These will be updated by next accelerometer reading
          y: 0,

```

```

        z: 0,
        heartRate: heartRateService.lastHeartRate,
        batteryLevel: battery,
    ));
  }
}, onError: (e) => debugPrint('Battery stream error: $e'));

_stateStreamController.add(true);
} catch (e) {
  debugPrint('Start error: $e');
  _stateStreamController.add(false);
  if (!_isAccelerometerWorking) {
    // Only rethrow if accelerometer failed
    rethrow;
  }
}
}

Future<void> stop() async {
  debugPrint('Stopping BLE services...');
  await _accelSubscription?.cancel();
  _accelSubscription = null;
  _isAccelerometerWorking = false;
  _stateStreamController.add(false);
  debugPrint('All BLE services stopped');
}

void dispose() {
  debugPrint('Disposing BLE services...');
  stop();
  _dataStreamController.close();
  _stateStreamController.close();
  debugPrint('BLE services disposed');
}
}

// --- End of lib\accelerometer_service.dart ---

// --- Start of lib\battery.dart ---
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

class BatteryServiceException implements Exception {
  final String message;
  BatteryServiceException(this.message);
  @override
  String toString() => 'BatteryServiceException: $message';
}

class BatteryService {
  // Battery Service & Characteristic UUIDs
  static const String _batteryServiceUuid = '0000180f-0000-1000-8000-00805f9b34fb';
  static const String _batteryCharUuid = '00002a19-0000-1000-8000-00805f9b34fb';

```

```

int? _lastBatteryLevel;
final _dataStreamController = StreamController<int?>.broadcast();
StreamSubscription? _batterySubscription;

Stream<int?> get dataStream => _dataStreamController.stream;
int? get lastBatteryLevel => _lastBatteryLevel;

void _processBattery(List<int> value) {
  try {
    debugPrint('Raw battery data: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
    if (value.isEmpty) return;

    // Battery level is a single byte representing percentage (0-100)
    _lastBatteryLevel = value[0];
    debugPrint('Processed Battery Level: $_lastBatteryLevel%');
    _dataStreamController.add(_lastBatteryLevel);
  } catch (e) {
    debugPrint('Error processing battery data: $e');
    // Don't rethrow - continue processing future values
  }
}

Future<void> start(BluetoothDevice device) async {
  try {
    debugPrint('Setting up Battery service...');
    final services = await device.discoverServices();
    final batteryService = services.firstWhere(
      (s) => s.uuid.toString().toLowerCase() == _batteryServiceUuid.toLowerCase(),
      orElse: () => throw BatteryServiceException('Battery service not found'),
    );

    debugPrint('Found Battery service: ${batteryService.uuid}');
    final batteryChar = batteryService.characteristics.firstWhere(
      (c) => c.uuid.toString().toLowerCase() == _batteryCharUuid.toLowerCase(),
      orElse: () => throw BatteryServiceException('Battery characteristic not found'),
    );

    debugPrint('Found Battery characteristic: ${batteryChar.uuid}');
    debugPrint('Battery Properties: read=${batteryChar.properties.read},
notify=${batteryChar.properties.notify}');

    // Try initial read with timeout
    if (batteryChar.properties.read) {
      try {
        final value = await batteryChar.read()
          .timeout(const Duration(seconds: 2),
            onTimeout: () => throw TimeoutException('Battery read timeout'));
        debugPrint('Initial battery read successful');
        _processBattery(value);
      } catch (e) {
        debugPrint('Battery initial read failed: $e');
        // Continue even if initial read fails
      }
    }
  }
}

```

```

    }

    // Enable notifications if supported
    if (batteryChar.properties.notify) {
        try {
            final success = await batteryChar.setNotifyValue(true);
            debugPrint('Battery notifications enabled: $success');

            if (!success) {
                throw BatteryServiceException('Failed to enable battery notifications');
            }

            _batterySubscription = batteryChar.lastValueStream.listen(
                (value) {
                    try {
                        _processBattery(value);
                    } catch (e) {
                        debugPrint('Battery notification processing error: $e');
                        // Continue receiving notifications even if processing fails
                    }
                },
                onError: (e) {
                    debugPrint('Battery notification stream error: $e');
                    // Service will continue without battery updates
                },
            );
        } catch (e) {
            debugPrint('Error setting up battery notifications: $e');
            // Continue without notifications
        }
    } else {
        debugPrint('Battery characteristic does not support notify');
    }
} catch (e) {
    debugPrint('Battery service start failed: $e');
    _lastBatteryLevel = null;
    rethrow;
}
}

Future<void> stop() async {
    debugPrint('Stopping Battery service...');
    await _batterySubscription?.cancel();
    _batterySubscription = null;
    _lastBatteryLevel = null;
    debugPrint('Battery service stopped');
}

void dispose() {
    debugPrint('Disposing Battery service...');
    stop();
    _dataStreamController.close();
    debugPrint('Battery service disposed');
}

```

```

}
// --- End of lib\battery.dart ---

// --- Start of lib\heartrate.dart ---
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

class HeartRateServiceException implements Exception {
  final String message;
  HeartRateServiceException(this.message);
  @override
  String toString() => 'HeartRateServiceException: $message';
}

class HeartRateService {
  // Heart Rate Service & Characteristic UUIDs
  static const String _heartRateServiceUuid = '0000180d-0000-1000-8000-00805f9b34fb';
  static const String _heartRateCharUuid = '00002a37-0000-1000-8000-00805f9b34fb';

  int? _lastHeartRate;
  final _dataStreamController = StreamController<int?>.broadcast();
  StreamSubscription? _heartRateSubscription;

  Stream<int?> get dataStream => _dataStreamController.stream;
  int? get lastHeartRate => _lastHeartRate;

  void _processHeartRate(List<int> value) {
    try {
      debugPrint('Raw heart rate data: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
      if (value.isEmpty) return;

      // First byte contains flags
      final flags = value[0];
      final isFormat16Bit = (flags & 0x01) != 0;

      // Heart rate measurement value format
      if (value.length >= 2) {
        if (isFormat16Bit && value.length >= 3) {
          // 16-bit format
          _lastHeartRate = value[1] | (value[2] << 8);
        } else {
          // 8-bit format
          _lastHeartRate = value[1];
        }
        debugPrint('Processed Heart Rate: $_lastHeartRate BPM');
        _dataStreamController.add(_lastHeartRate);
      }
    } catch (e) {
      debugPrint('Error processing heart rate data: $e');
      // Don't rethrow - continue processing future values
    }
  }
}

```



```

Future<void> start(BluetoothDevice device) async {
  try {
    debugPrint('Setting up Heart Rate service...');
    final services = await device.discoverServices();
    final heartRateService = services.firstWhere(
      (s) => s.uuid.toString().toLowerCase() == _heartRateServiceUuid.toLowerCase(),
      orElse: () => throw HeartRateServiceException('Heart rate service not found'),
    );

    debugPrint('Found Heart Rate service: ${heartRateService.uuid}');
    final heartRateChar = heartRateService.characteristics.firstWhere(
      (c) => c.uuid.toString().toLowerCase() == _heartRateCharUuid.toLowerCase(),
      orElse: () => throw HeartRateServiceException('Heart rate characteristic not
found'),
    );

    debugPrint('Found Heart Rate characteristic: ${heartRateChar.uuid}');
    debugPrint('Heart Rate Properties: read=${heartRateChar.properties.read},
notify=${heartRateChar.properties.notify}');

    // Heart rate measurement only supports notifications
    if (heartRateChar.properties.notify) {
      try {
        final success = await heartRateChar.setNotifyValue(true);
        debugPrint('Heart Rate notifications enabled: $success');

        if (!success) {
          throw HeartRateServiceException('Failed to enable heart rate
notifications');
        }

        _heartRateSubscription = heartRateChar.lastValueStream.listen(
          (value) {
            try {
              _processHeartRate(value);
            } catch (e) {
              debugPrint('Heart rate notification processing error: $e');
              // Continue receiving notifications even if processing fails
            }
          },
          onError: (e) {
            debugPrint('Heart rate notification stream error: $e');
            // Service will continue without heart rate updates
          },
        );
      } catch (e) {
        debugPrint('Error setting up heart rate notifications: $e');
        // Continue without notifications
      }
    } else {
      throw HeartRateServiceException('Heart rate characteristic does not support
notifications');
    }
  }
}

```

```

    } catch (e) {
      debugPrint('Heart rate service start failed: $e');
      _lastHeartRate = null;
      rethrow;
    }
  }

Future<void> stop() async {
  debugPrint('Stopping Heart Rate service...');
  await _heartRateSubscription?.cancel();
  _heartRateSubscription = null;
  _lastHeartRate = null;
  debugPrint('Heart Rate service stopped');
}

void dispose() {
  debugPrint('Disposing Heart Rate service...');
  stop();
  _dataStreamController.close();
  debugPrint('Heart Rate service disposed');
}
}
// --- End of lib\heartrate.dart ---

// --- Start of lib\main.dart ---
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';
import 'package:permission_handler/permission_handler.dart';
import 'accelerometer_service.dart';
import 'battery.dart';
import 'heartrate.dart';
import 'widgets/accelerometer_widget.dart';
import 'widgets/battery_widget.dart';
import 'widgets/heart_rate_widget.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  FlutterBluePlus.setLogLevel(LogLevel.verbose, color: true);
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'CL837 Sensor Display',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        visualDensity: VisualDensity.adaptivePlatformDensity,
      ),
    ),
  },

```

```

        home: const SensorDisplayPage(),
    );
}
}

class SensorDisplayPage extends StatefulWidget {
  const SensorDisplayPage({super.key});

  @override
  State<SensorDisplayPage> createState() => _SensorDisplayPageState();
}

class _SensorDisplayPageState extends State<SensorDisplayPage> {
  final SensorService _sensorService = SensorService();
  final HeartRateService _heartRateService = HeartRateService();
  final BatteryService _batteryService = BatteryService();
  static const String targetDeviceName = 'CL837-0753644';

  BluetoothDevice? connectedDevice;
  SensorData? latestData;
  bool isScanning = false;
  bool isConnecting = false;
  late StreamSubscription<SensorData> _dataSubscription;

  @override
  void initState() {
    super.initState();
    _initializeBluetooth();
  }

  Future<void> _initializeBluetooth() async {
    await _requestPermissions();
    await FlutterBluePlus.turnOn();
  }

  Future<void> _requestPermissions() async {
    await Future.wait([
      Permission.bluetooth.request(),
      Permission.bluetoothScan.request(),
      Permission.bluetoothConnect.request(),
      Permission.location.request(),
    ]);
  }

  Future<void> startScan() async {
    if (isScanning) return;

    setState(() {
      isScanning = true;
    });

    try {
      await FlutterBluePlus.startScan(timeout: const Duration(seconds: 5));
    }
  }
}

```

```

FlutterBluePlus.scanResults.listen((results) {
  for (ScanResult r in results) {
    debugPrint('Found device: ${r.device.name}');
    if (r.device.name == targetDeviceName) {
      connectToDevice(r.device);
      FlutterBluePlus.stopScan();
      break;
    }
  }
});

await Future.delayed(const Duration(seconds: 5));

if (mounted) {
  setState(() {
    isScanning = false;
  });
}
} catch (e) {
  debugPrint('Error during scan: $e');
  if (mounted) {
    setState(() {
      isScanning = false;
    });
  }
  showError('Error during scan: $e');
}
}

Future<void> connectToDevice(BluetoothDevice device) async {
  if (isConnecting) return;
  setState(() {
    isConnecting = true;
  });

  try {
    await device.connect(timeout: const Duration(seconds: 10));

    try {
      await _sensorService.start(device, _heartRateService, _batteryService);
    } catch (e) {
      if (!_sensorService.isAccelerometerWorking) {
        showError('Critical error: Accelerometer not working');
        await disconnectDevice();
        return;
      } else {
        showWarning('Some sensors may not work properly. Accelerometer is still functional.');
```

```

        latestData = data;
    });
},
onError: (error) {
    debugPrint('Sensor data stream error: $error');
    showWarning('Some sensor data may be temporarily unavailable');
},
);

if (mounted) {
    setState(() {
        connectedDevice = device;
        isConnected = false;
    });
}
} catch (e) {
    debugPrint('Connection error: $e');
    if (mounted) {
        setState(() {
            isConnected = false;
        });
    }
    showError('Failed to connect to device');
}
}

Future<void> disconnectDevice() async {
    try {
        await _dataSubscription.cancel();
        await _sensorService.stop();
        await _heartRateService.stop();
        await _batteryService.stop();
        await connectedDevice?.disconnect();
    } catch (e) {
        debugPrint('Error during disconnect: $e');
    }
    if (mounted) {
        setState(() {
            connectedDevice = null;
            latestData = null;
        });
    }
}

@override
void dispose() {
    _dataSubscription.cancel();
    _sensorService.dispose();
    _heartRateService.dispose();
    _batteryService.dispose();
    disconnectDevice();
    super.dispose();
}

```

```

void showWarning(String message) {
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(message),
      backgroundColor: Colors.orange,
      duration: const Duration(seconds: 3),
    ),
  );
}

void showError(String message) {
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(message),
      backgroundColor: Colors.red,
      duration: const Duration(seconds: 3),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('CL837 Sensor Display'),
      actions: [
        if (connectedDevice != null)
          IconButton(
            icon: const Icon(Icons.bluetooth_connected),
            tooltip: 'Disconnect',
            onPressed: disconnectDevice,
          )
      ],
    ),
    body: Column(
      children: [
        Container(
          color: Colors.grey[100],
          padding: const EdgeInsets.all(16.0),
          child: Row(
            children: [
              Expanded(
                child: ElevatedButton.icon(
                  icon: const Icon(Icons.search),
                  label: Text(
                    isScanning ? 'Scanning...' :
                    isConnecting ? 'Connecting...' :
                    'Scan for Device',
                  ),
                onPressed: (isScanning || isConnecting) ? null : startScan,
              ),
            ],
          ),
        ),
      ],
    ),
  );
}

```

```

    ],
  ),
),
Expanded(
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        HeartRateWidget(latestData: latestData),
        const SizedBox(height: 8),
        BatteryWidget(latestData: latestData),
        const SizedBox(height: 16),
        AccelerometerWidget(latestData: latestData),
      ],
    ),
  ),
),
),
],
),
);
}
}
// --- End of lib\main.dart ---

```

```

// --- Start of lib\widgets\accelerometer_widget.dart ---

```

```

import 'package:flutter/material.dart';
import '../accelerometer_service.dart';

```

```

class AccelerometerWidget extends StatelessWidget {
  final SensorData? latestData;

  const AccelerometerWidget({Key? key, required this.latestData}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        const Text(
          'Accelerometer',
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 8),
        if (latestData != null) ...[
          _buildAxisRow('X', latestData!.x, Colors.red),
          const SizedBox(height: 8),
          _buildAxisRow('Y', latestData!.y, Colors.green),
          const SizedBox(height: 8),
          _buildAxisRow('Z', latestData!.z, Colors.blue),
        ] else
      ]
    );
  }
}

```

```

    const Center(
      child: Text(
        'Waiting for data...',
        style: TextStyle(
          fontSize: 16,
          color: Colors.grey,
        ),
      ),
    ),
  ],
);
}

```

```

Widget _buildAxisRow(String axis, double value, Color color) {
  return Card(
    elevation: 2,
    child: Padding(
      padding: const EdgeInsets.all(12.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Container(
                width: 24,
                height: 24,
                decoration: BoxDecoration(
                  color: color.withOpacity(0.2),
                  borderRadius: BorderRadius.circular(4),
                ),
                child: Center(
                  child: Text(
                    axis,
                    style: TextStyle(
                      color: color,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ),
              ),
              const SizedBox(width: 12),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    LinearProgressIndicator(
                      value: (value + 8.0) / 16.0, // Normalize from -8g to 8g
                      backgroundColor: color.withOpacity(0.1),
                      valueColor: AlwaysStoppedAnimation<Color>(color),
                    ),
                    const SizedBox(height: 4),
                    Text(
                      '${value.toStringAsFixed(3)} g',
                      style: TextStyle(

```



```

        color: color,
        fontWeight: FontWeight.w500,
      ),
    ),
  ],
),
),
],
),
],
),
),
);
}
}

// --- End of lib\widgets\accelerometer_widget.dart ---

// --- Start of lib\widgets\battery_widget.dart ---
import 'package:flutter/material.dart';
import '../accelerometer_service.dart';

class BatteryWidget extends StatelessWidget {
  final SensorData? latestData;

  const BatteryWidget({Key? key, required this.latestData}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return _buildStatusRow(
      'Battery',
      latestData?.batteryLevel != null ? '${latestData!.batteryLevel}%' : 'N/A',
      Icons.battery_full,
      color: _getBatteryColor(latestData?.batteryLevel),
    );
  }

  Widget _buildStatusRow(String label, dynamic value, IconData icon, {Color? color}) {
    return Card(
      elevation: 2,
      child: Padding(
        padding: const EdgeInsets.all(12.0),
        child: Row(
          children: [
            Icon(icon, size: 24, color: color),
            const SizedBox(width: 12),
            Text(
              label,
              style: const TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.bold,
              ),
            ),
            const Spacer(),
          ],
        ),
      ),
    );
  }
}

```

```

        Text(
          value?.toString() ?? 'N/A',
          style: TextStyle(
            fontSize: 16,
            color: color,
            fontWeight: FontWeight.bold,
          ),
        ),
      ],
    ),
  ),
);
}

Color _getBatteryColor(int? level) {
  if (level == null) return Colors.grey;
  if (level > 60) return Colors.green;
  if (level > 30) return Colors.orange;
  return Colors.red;
}
}

// --- End of lib\widgets\battery_widget.dart ---

// --- Start of lib\widgets\heart_rate_widget.dart ---
import 'package:flutter/material.dart';
import '../accelerometer_service.dart';

class HeartRateWidget extends StatelessWidget {
  final SensorData? latestData;

  const HeartRateWidget({Key? key, required this.latestData}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return _buildStatusRow(
      'Heart Rate',
      latestData?.heartRate != null ? '${latestData!.heartRate} BPM' : 'N/A',
      Icons.favorite,
      color: Colors.red,
    );
  }

  Widget _buildStatusRow(String label, dynamic value, IconData icon, {Color? color}) {
    return Card(
      elevation: 2,
      child: Padding(
        padding: const EdgeInsets.all(12.0),
        child: Row(
          children: [
            Icon(icon, size: 24, color: color),
            const SizedBox(width: 12),
            Text(
              label,

```

```

        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
        ),
      ),
      const Spacer(),
      Text(
        value?.toString() ?? 'N/A',
        style: TextStyle(
          fontSize: 16,
          color: color,
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
);
}
}

// --- End of lib\widgets\heart_rate_widget.dart ---

```